Informatik 11
Embedded Software

RWTH AACHEN UNIVERSITY

Praktikum Systemprogrammierung

Versuch 1

Mikrocontroller & AD/DA-Wandlung

Lehrstuhl Informatik 11 - RWTH Aachen

7. Mai 2026

Commit: Unknown

Inhaltsverzeichnis

1 Mikrocontroller & AD/DA-Wandlung **3**
1.1 Grundlagen . . . 4
1.1.1 Einführung in die Verwendung des ATmega644 . . . 4
1.1.2 AD/DA-Wandlung . . . 9
1.2 Hausaufgaben . . . 15
1.2.1 Allgemeines . . . 16
1.2.2 Aufgaben zur AD/DA-Wandlung . . . 16
1.2.3 Aufgaben zum ATmega644 . . . 18
1.2.4 Zusammenfassung . . . 28
1.3 In Präsenz . . . 30
1.3.1 Versuche zur AD/DA-Wandlung . . . 30
1.3.2 Versuche zum ATmega644 . . . 31
1.3.3 Anwendungsaufgaben . . . 32
1.4 Pinbelegung . . . 33

Dieses Dokument ist Teil der begleitenden Unterlagen zum *Praktikum Systemprogrammierung*. Alle zu diesem Praktikum benötigten Unterlagen stehen im Moodle-Lernraum unter https://moodle.rwth-aachen.de zum Download bereit. Für Kritik, Anregungen oder Verbesserungsvorschläge können Sie gerne Feedback über den Moodle-Lernraum hinterlassen oder eine E-Mail an folgende Adresse senden:

support.psp@embedded.rwth-aachen.de

Nennen Sie dabei bitte den Namen der Anwendungsaufgabe, auf die sich Ihr Feedback bezieht.

1 Mikrocontroller & AD/DA-Wandlung

Das Lernziel dieses Versuchs ist das Verständnis der folgenden Zusammenhänge:

- Umgang mit der Entwicklungsumgebung Microchip Studio 7
- Grundlagen der Programmiersprache C
- Programmierung eines Mikrocontrollers
- AD/DA-Wandlung
- Verwendung von Interrupts

Hinweis

Die Dokumente im Praktikum Systemprogrammierung bestehen aus zwei Teilen:

- **Grundlagen**
- **Hausaufgaben** und **In Präsenz**

Für die Bearbeitung der Aufgaben werden Teile der Theorie benötigt, die im Grundlagenkapitel vorgestellt wird. Es wird außerdem empfohlen, bei Unklarheiten das *Begleitende Dokument* zu konsultiern.

1.1 Grundlagen

Mikrocontroller sind Halbleiterchips, die neben dem Prozessor weitere Peripheriefunktionen auf einem Chip vereinen. Dadurch kommen Projekte mit Mikrocontrollern oft mit wenigen Bauteilen aus. Sie sind häufig in alltäglichen Gegenständen zu finden und werden dann eingebettete Systeme genannt. Sie finden insbesondere in der digitalen Kontrolle und Steuerung von Geräten und Prozessen Anwendung. Ihre Leistung und Ausstattung werden für ihren jeweilige Einsatzzweck angepasst, wodurch die Kosten, das Format und die Leistungsaufnahme optimiert werden können. Das Programmieren von Mikrocontrollern stellt, verglichen mit dem herkömmlicher Computersysteme, zusätzliche Anforderungen an die Entwickler. Dazu zählt, dass hardwarenahe Programmiersprachen wie C oder Assembler benutzt werden, sowie der Umgang mit Ressourcenknappheit.

1.1.1 Einführung in die Verwendung des ATmega644

Im Folgenden werden zunächst grundlegende Informationen zur Softwareentwicklung für eingebettete Systeme vorgestellt. Anschließend wird das Evaluationsboard und der darauf platzierte 8-Bit Mikrocontroller ATmega644 eingeführt.

Softwareentwicklung für Microchip-Mikroprozessoren

Software für Mikroprozessoren der Firma Microchip kann in verschiedenen Programmiersprachen entwickelt werden. Im Rahmen des Praktikums Systemprogrammierung wird ausschließlich die Sprache C mit einzelnen Assembleranweisungen verwendet.
Ein C Programm für einen Mikrocontroller unterscheidet sich nicht grundlegend von einem Programm für ein übliches Computersystem. Es wird sequenziell abgearbeitet und verfügt über eine sogenannte `main`-Methode, welche zu Beginn der Ausführung automatisch aufgerufen wird und den Programmeinstiegspunkt darstellt.
Um ein Programm auf den Mikrocontroller zu übertragen und zu debuggen wird ein sogenannter *JTAG Programmierer* verwendet. Als geeignete Entwicklungsumgebung stellt Microchip das Microchip Studio 7 kostenlos zur Verfügung. Einen entsprechenden Installer finden Sie im Moodle-Lernraum des Praktikums. Für weitere Informationen zu Microchip Studio 7 und C sei auf die Kapitel *3: Einführung in Microchip Studio 7* und *5: Einführung in die C Programmierung* im *Begleitenden Dokument* verwiesen.

Informationen zum Evaluationsboard des ATmega644

Im Praktikum Systemprogrammierung wird ein 8-Bit Mikrocontroller der Firma Microchip verwendet – der ATmega644. Dieser Mikrocontroller ist auf dem Evaluationsboard des Praktikums mit 20 MHz getaktet und besitzt verschiedene interne Komponenten, wie zum Beispiel EEPROM-Speicher, AD/DA-Wandler, vier I/O-Ports, eine USART Schnittstelle und drei Timer. Darüber hinaus verfügt er über einen Interrupt-Controller. Das heißt, dass unabhängig vom aktuell ausgeführten Code Ereignisse erkannt und auf diese reagiert werden kann (d.h. ein *Interrupt* eintritt), indem die Ausführung

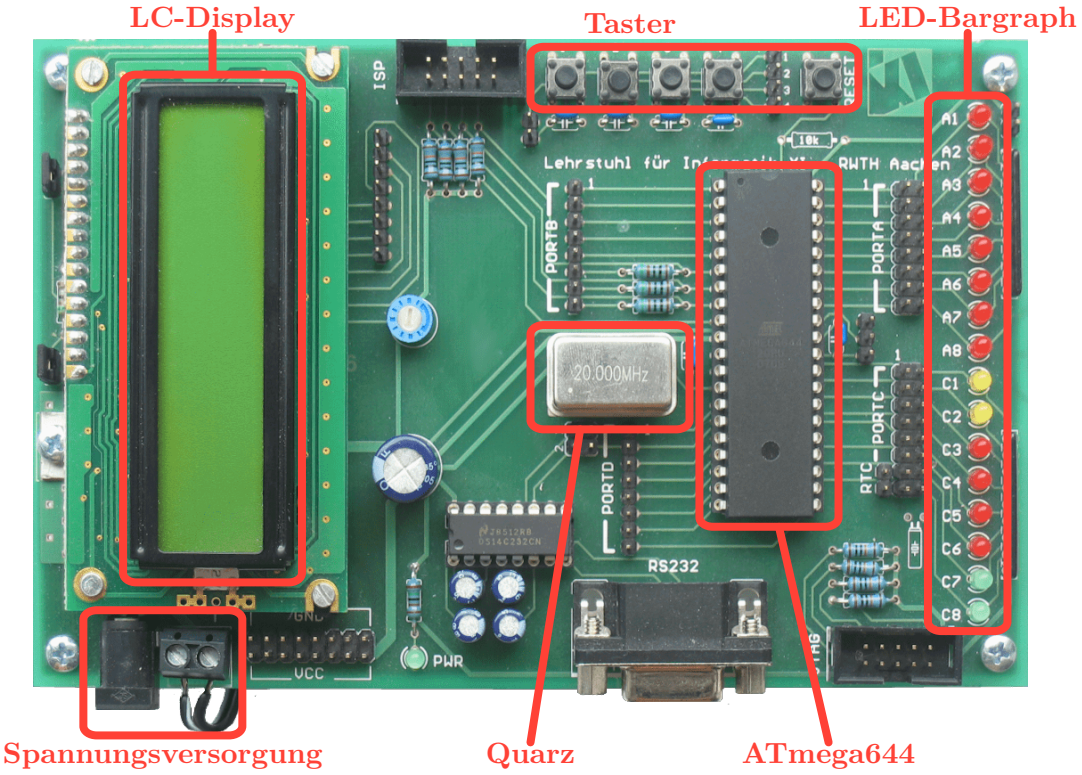


Abbildung 1.1: Das Evaluationsboard des ATmega644

des Hauptprogramms unterbrochen wird. Diese Ereignisse können extern auftreten, beispielsweise in Form einer Pegeländerung an einem Pin, oder intern, beispielsweise wenn ein spezielles Register einen bestimmten Wert erreicht hat.

Ein Evaluationsboard ist eine Platine, die es erlaubt, einen Mikrocontroller bequem programmieren und debuggen zu können, ohne zuerst eine Schaltung für diesen zu entwickeln. Das Evaluationsboard des Lehrstuhls für Informatik 11 der RWTH Aachen (Abbildung 1.1) bietet eine komfortable Arbeitsumgebung für den Umgang mit dem Mikrocontroller und stellt Pinleisten bereit, über die dieser leicht mit Eingabe- und Ausgabegeräten (LCD, Buttons, LEDs etc.) verbunden werden kann.

Hinweis

Eine ausführlichere Beschreibung zu den Komponenten und deren Verwendung kann Kapitel *2.1: Der Mikrocontroller ATmega644* im *Begleitenden Dokument* entnommen werden. Eine technische Beschreibung des ATmega644 befindet sich im *ATmega644 Datasheet*.

Konfiguration von Ports

Der ATmega644 verfügt über 40 Pins, von denen 32 als Ein-/Ausgabe Pins verwendet werden können. Ist ein Pin als Ausgang konfiguriert, so kann der Mikrocontroller die Spannung 5 V (logisch 1) oder 0 V (logisch 0) ausgeben. Mit Hilfe eines als Eingang konfigurierten Pins kann der Mikrocontroller feststellen, ob an diesem Pin ein Spannungspegel von 5 V oder 0 V angelegt wurde.
Die Pins sind auf vier Organisationseinheiten aufgeteilt: Die Ports A, B, C und D (siehe Abbildung 1.1). In diesen Organisationseinheiten befinden sich jeweils 8 Ein-/Ausgabe Pins, wobei diese von 0 bis 7 durchnummeriert sind. Jeder dieser Pins kann über spezielle Prozessorregister konfiguriert werden. Für jeden Port existieren sogenannte *Data Direction-*, *Port Output-* und *Port Input-*Register (DDR, PORT und PIN), welche der Konfiguration der Pins dienen. Diese können analog zu Variablen ausgelesen und verändert werden, wie im folgenden Abschnitt exemplarisch an Port A gezeigt wird.

Data Direction Register Das *Port A Data Direction Register* (DDRA) steuert, welche Pins eines Ports zur Ein- und welche zur Ausgabe verwendet werden. Eine 0 des least significant Bits, auch Stelle DDRA0 genannt (Analog dazu DDRA1 für das second-least significant Bit usw.), im Register DDRA bedeutet beispielsweise, dass der erste Pin von Port A als Eingang verwendet wird. Eine 1 an dieser Stelle konfiguriert den Pin als Ausgang.

Port Data Register Das *Port A Data Register* (PORTA) hat verschiedene Funktionen, abhängig davon, ob ein Pin durch das entsprechende Data Direction Register als Ein- oder als Ausgang definiert wurde. Wenn ein Pin als Ausgabepin definiert wurde, wird PORTA dazu verwendet, um zu steuern, welches Signal ausgegeben werden soll. Wenn ein Pin als Eingabepin definiert wurde, wird PORTA dazu verwendet, den Pullup-Widerstand für diesen Pin zu aktivieren oder zu deaktivieren.

Port Input Pins Das *Port A Input Pins-Register* PINA wird zum Auslesen externer Signale verwendet.

DDR	PORT	PIN
0: Eingang	Pullup-Widerstand	anliegender Wert
1: Ausgang	zu schreibender Wert	-

Tabelle 1.1: Register

Eine ausführlichere Beschreibung der Register inklusive Beispiele kann im Kapitel *2.1.1: I/O-Ports* im *Begleitenden Dokument* nachgeschlagen werden. Zur Konfiguration dieser Register ist die Manipulation einzelner Bits nötig. Da es nur möglich ist ganze Bytes zu adressieren nutzt man Bitmasken um einzelne Bits zu manipulieren ohne den Rest vom Byte zu verändern. Bitmasken sind Zahlenwerte, die oft in binärer oder hexadezimaler Schreibweise angegeben werden. Dabei werden führende Nullen mit angegeben, um die Größe des Ergebnisses zu verdeutlichen. Mit Hilfe binärer Operatoren wie & und | lassen

sich dadurch einzelne Bits eines Registers verändern, ohne dass die restlichen Bits davon betroffen sind. Für weitere Informationen kann das Kapitel *5.1.16: Registerzugriff, Bitshifting & Bitmasken* im *Begleitenden Dokument* herangezogen werden.

Buttons

In diesem Praktikum werden Buttons verwendet, um Eingaben an ein auf dem Mikrocontroller laufendem Programm tätigen zu können. Das Evaluationsboard verfügt über vier Buttons, die beliebig mit Pins verkabelt werden können. Des Weiteren kommt in diesem Versuch eine spezielle Platine zum Einsatz, die für die Realisierung des manuellen AD/DA-Wandlers nötig ist (siehe Abbildung 1.2). Diese verfügt ebenfalls über einen Button, welcher fest mit Pin C1 verbunden ist.
Um den Zustand eines Buttons auslesen zu können, muss zunächst der entsprechende Pin als Eingang konfiguriert werden. Bei Buttons ist es zudem notwendig, sogenannte Pullup-Widerstände für die jeweils genutzten Pins zu aktivieren, um zwischen einem gedrückten und nicht-gedrückten Zustand unterscheiden zu können. Im Anschluss daran kann der Zustand des Buttons aus dem zugehörigen PIN-Register ausgelesen werden. Hierbei ist zu beachten, dass das entsprechende Bit im PIN-Register im gedrückten Zustand den Wert 0 und im nicht-gedrückten Zustand den Wert 1 hat.

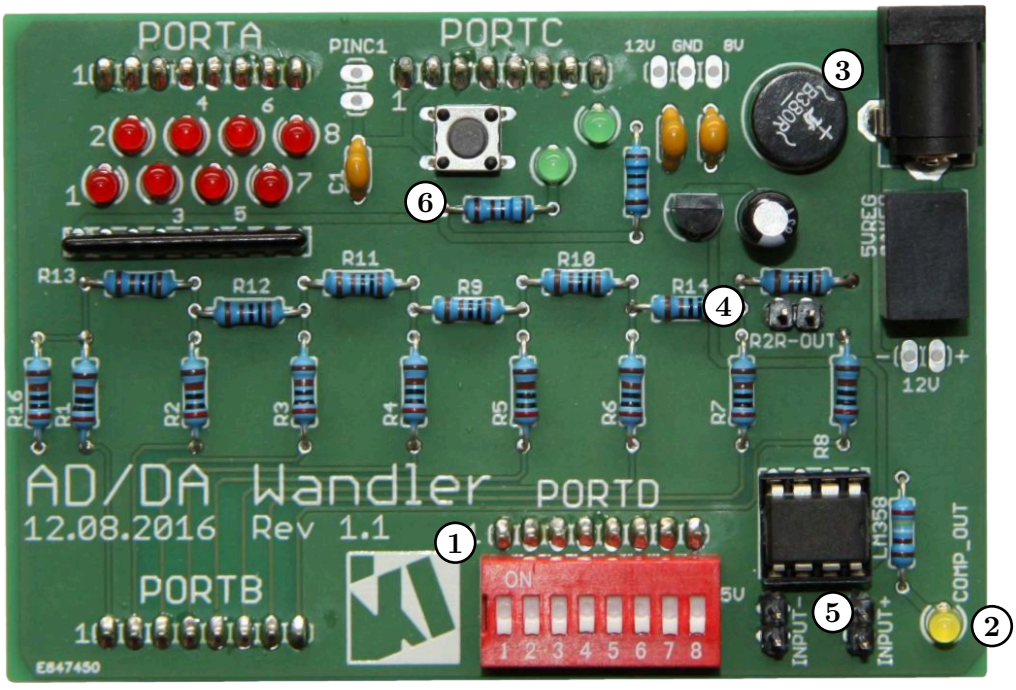


Abbildung 1.2: Platine mit Hardware zur AD/DA-Wandlung

Interrupts

Ein Interrupt ist eine automatische, ereignisgesteuerte Unterbrechung des Programmablaufs. Der ATmega644 verfügt über eine Vielzahl von konfigurierbaren Interrupts. Eine Möglichkeit, Interrupts intern auszulösen, stellen Timer dar. Sie können beispielsweise so konfiguriert werden können, dass sie in definierten Zeitabständen einen Interrupt auslösen.
Um auf Interrupts zu reagieren, müssen sogenannte Interrupt Service Routinen (ISRs) implementiert werden, die als Behandlungsfunktionen für diese Ereignisse fungieren. Nach Abarbeitung einer ISR wird der normale Programmablauf an der Stelle fortgesetzt, an der dieser durch den Aufruf der ISR unterbrochen wurde.

LC-Display

Zusammen mit den Versuchsunterlagen wird ein Treiber für das LC-Display des Evaluationsboards zur Verfügung gestellt. Folgende Schnittstellenfunktionen des LCD-Treibers sind verfügbar:

Befehl	Bedeutung	Beispiel
lcd_init(void)	Initialisierung	lcd_init();
lcd_clear(void)	Löscht das Display. Das nächste Zeichen wird in der ersten Spalte und Zeile ausgegeben.	lcd_clear();
lcd_writeChar(char)	Gibt ein einzelnes Zeichen aus.	lcd_writeChar('!');
lcd_writeDec(uint16_t)	Gibt eine Ganzzahl ohne führende Nullen aus.	lcd_writeDec(471);
lcd_writeString(const char*)	Gibt eine Zeichenkette aus.	lcd_writeString("xy");
lcd_writeProgString(const char*)	Gibt eine Zeichenkette aus dem Flash-Speicher aus.	lcd_writeProgString(PSTR("xy"));

Tabelle 1.2: Ausgewählte Funktionen des LCD-Treibers

In diesem Versuch wird die Funktion lcd_writeVoltage(uint16_t, uint16_t, uint8_t) benötigt, um einen diskreten Spannungswert in einen Spannungswert in Gleitkommadarstellung umzuwandeln. Die Funktion kann folgendermaßen verwendet werden: Der auszugebende Spannungswert wird als diskreter Wert im ersten Parameter übergeben, während der zweite Parameter die obere Schranke des ersten Parameters angibt. Der dritte Parameter gibt die obere Schranke der Spannung an. lcd_writeVoltage(113, 255, 5) erzeugt beispielsweise die Ausgabe $\frac{113}{255} * 5\,\mathrm{V} =$„2.215V“.

? Lernerfolgsfragen

- Was muss vor der ersten Verwendung des Displays sichergestellt werden?
- Wie kann das Display gelöscht werden? Wie wird es mit Buchstaben, Zahlen oder Zeichenketten beschrieben?

Debugging

Es wird empfohlen, das Kapitel *6.2: Debugging-Methoden* im *Begleitenden Dokument* zu lesen. In diesem werden Methoden vorgestellt, die bei einer möglichen Fehlersuche hilfreich sind. Dazu gehören beispielsweise Breakpoints oder das Auslesen des Speichers und der aktuellen Konfiguration der Prozessorregister.

Verkabelung des Evaluationsboards

Auf den letzten Seiten dieses und folgender Versuchsdokumente ist jeweils eine Übersicht über die Pinbelegung des aktuellen Versuchs zu finden. Zudem ist zu beachten, dass die Mikrocontroller im Testpool, falls nicht ausdrücklich anders angegeben, stets gemäß dieser Tabelle verkabelt sind.

1.1.2 AD/DA-Wandlung

In diesem Abschnitt werden zunächst grundlegende Informationen zum Thema Analog-Digital- bzw. Digital-Analog-Wandlung (AD/DA-Wandlung) vorgestellt. Anschließend werden Grundlagen zu Komparatoren und R-2R-Netzwerken erläutert. Beides sind elektrische Schaltungen, die für die in diesem Versuch vorgestellte AD/DA-Wandlung benötigt werden. Abschließend werden Methoden vorgestellt, um eine AD/DA-Wandlung durchzuführen.

AD/DA-Wandler

Ein Digital-Analog-Wandler (DA-Wandler, engl. Digital-to-Analog Converter (DAC)) konvertiert quantisierte digitale Signale in analoge Signale. Ein Analog-Digital Wandler (AD-Wandler, engl. Analog-to-Digital-Converter (ADC)) hingegen, setzt analoge Eingangssignale in digitale Daten um. AD/DA-Wandler werden unter anderem in der Messtechnik oder Audio-Video-Verarbeitung eingesetzt. In diesen Anwendungen liegen die Eingangsgrößen analog bzw. digital vor und sollen in das jeweils andere Format konvertiert werden. Ein Beispiel hierfür ist die Aufnahme und Wiedergabe von Musik auf einem Computer.

Um ein zeit- und wertdiskretes digitales Signal aus einem kontinuierlichen analogen Signal zu erzeugen, wird das Ursprungssignal zunächst an festen Zeitpunkten abgetastet. Die Abtastung einer analogen Spannung benötigt eine bestimmte Zeitspanne, da eine AD-Wandlung in den meisten Fällen durch eine iterative Approximation der zu messenden Spannung realisiert wird. Je kürzer diese Zeitspanne ist, desto größer ist die sogenannte *Umsetzungsgeschwindigkeit* des Wandlers.
Ist die Umsetzungsgeschwindigkeit eines Wandlers zu niedrig und damit die Abtastzeitpunkte zu weit voneinander entfernt, kann der ursprüngliche Signalverlauf nicht aus dem digitalen Signal rekonstruiert werden, was als *Alias-Effekt* bezeichnet wird. Um dies zu verhindern ist nach dem Abtasttheorem von Nyquist und Shannon ein Wandler mit einer Umsetzungsgeschwindigkeit nötig, welche mindestens doppelt so hoch ist wie die maximale Frequenz des abzutastenden Signals.
Im Anschluss an die Abtastung müssen die abgetasteten analogen Spannungswerte

auf diskrete Werte gerundet werden. Dieser Vorgang heißt *Quantisierung*. Die dabei entstehenden diskreten Werte heißen Quantisierungsstufen. Je mehr Quantisierungsstufen zur Verfügung stehen, desto geringer ist der auftretende Rundungsfehler und desto genauer wird das ursprüngliche Signal repräsentiert. Die Quantisierungsstufen werden üblicherweise in dem erwarteten Wertebereich des analogen Signals gleichverteilt. Nach der Abtastung und Quantisierung können die Signalwerte codiert werden, indem jeder Quantisierungsstufe eine Zahl zugeordnet wird. Das analoge, zeitkontinuierliche Ursprungssignal wird somit durch eine digitale, zeitdiskrete Folge von Zahlen repräsentiert.

Beispiel Bei einer Quantisierung von 5 V auf 256 Werte erhält man eine Quantisierungsstufe von $\frac{5\,\mathrm{V}}{256} \approx 20\,\mathrm{mV}$. Die Codierung der Binärzahl 0b01111011 (dezimal 123) in eine analoge Spannung ist demnach rund $2{,}46\,\mathrm{V} = 123 \cdot 20\,\mathrm{mV}$.

Die wichtigsten Kenngrößen eines AD/DA-Wandlers sind seine Auflösung und die Umsetzungsgeschwindigkeit. Die Auflösung gibt an, wie viele Bits zur Codierung der Quantisierungsstufen zur Verfügung stehen. Eine Auflösung von 8 b bedeutet beispielsweise, dass $2^8 = 256$ Quantisierungsstufen zur Darstellung existieren. Dies entspricht einer Genauigkeit von 0,39 % ($\frac{1}{256}$) bezogen auf den Messbereich.

? Lernerfolgsfragen

- Wozu dienen AD/DA-Wandler?
- Was passiert bei der Abtastung eines Signals?
- Was passiert bei der Quantisierung eines Signals?
- Welche sind die wichtigsten Kenngrößen von AD/DA-Wandlern?

Komparator

Durch die in Abbildung 1.3 gegebene Verschaltung kann ein Komparator mit Hilfe eines Operationsverstärkers konstruiert werden. Am Ausgang des Komparators steht ein Signal zur Verfügung, das anzeigt, welche der beiden Eingangsspannungen U_e bzw.

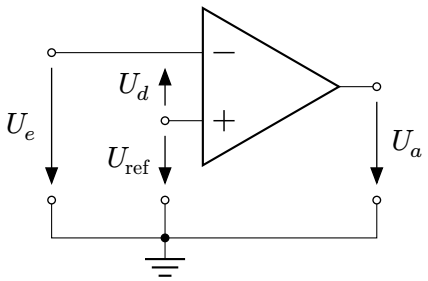


Abbildung 1.3: Komparatorschaltung

U_{ref} höher ist. Ist die Spannung am positiven Eingang höher als die Spannung am negativen Eingang, so entspricht die Ausgangsspannung annähernd der positiven Versorgungsspannung. Im gegenteiligen Fall ($U_{\text{ref}} < U_e$) entspricht die Ausgangsspannung der negativen Versorgungsspannung. Somit können selbst minimale Unterschiede der beiden Eingangsspannungen erkannt werden. Um die Ausgangsspannung des Komparators einfach ablesen zu können, befindet sich auf dem AD/DA-Board eine mit COMP_OUT beschriftete LED, an der die Ausgangsspannung anliegt.

R-2R-Netzwerk

Die einfachste Methode einer DA-Wandlung ist ein R-2R-Netzwerk. Dies ist das einzige in diesem Versuch genutzte Verfahren für die Umwandlung von digitalen Signalen in analoge Spannungen. Ein R-2R-Netzwerk verwendet nur Widerstände der Größen R und $2R$. Abbildung 1.4 zeigt die Verschaltung eines n-Bit R-2R-Netzwerks. An den Eingängen $b_0, ..., b_{n-1}$ liegt jeweils zur Darstellung einer digitalen Eins die Betriebsspannung an, zur Darstellung einer digitalen Null werden die Eingänge auf Masse gelegt.

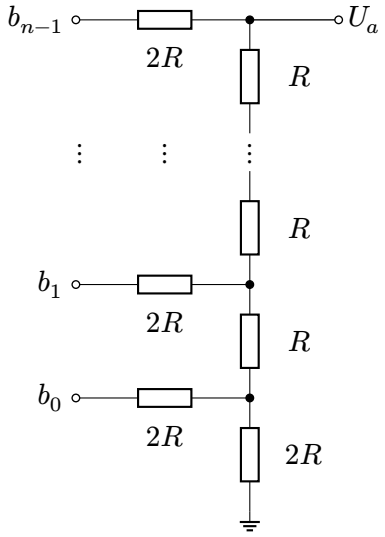


Abbildung 1.4: R-2R Widerstandsnetzwerk

Die Ausgangsspannung U_a kann wie folgt berechnet werden:

$$U_a = V_{\text{cc}} \cdot \left(\frac{b_{n-1}}{2^1} + \frac{b_{n-2}}{2^2} + \frac{b_{n-3}}{2^3} + ... + \frac{b_0}{2^n} \right)$$

Hierbei stellt b_i jeweils den logischen Wert des Eingangs i dar. Durch unterschiedliche Beschaltung der Eingänge $b_0, ..., b_{n-1}$ kann die analoge Ausgangsspannung U_a variiert werden.

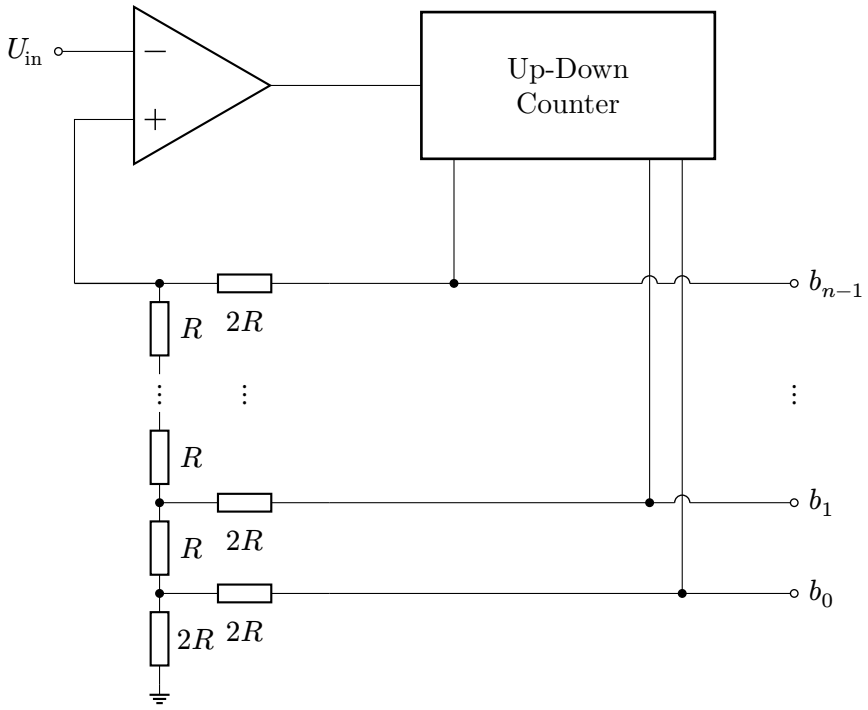


Abbildung 1.5: Tracking-Wandler

Tracking-Wandler

Der Tracking-Wandler ist ein Analog-Digital-Wandler, der auf eine von einem DA-Wandler erzeugte Referenzspannung zurückgreift, um die zu messende Spannung zu ermitteln. Die quantisierte Referenzspannung wird dabei in einer n-Bit Variable r festgehalten, die zu Beginn mit einem beliebigen Wert initialisiert werden kann. Der interne Aufbau eines Tracking-Wandlers ist in Abbildung 1.5 dargestellt und wird im Folgenden erläutert. Die generierte Referenzspannung wird mit der zu messenden Spannung mit Hilfe eines Komparators verglichen. Ist die Messspannung höher als die Referenzspannung (Komparator $K = 0$), inkrementiert ein Up-Down-Counter die Variable r, andernfalls wird r dekrementiert. Nach Inkrementierung bzw. Dekrementierung wird der neue Wert von r als neue Referenzspannung ausgegeben und das Ergebnis des Komparators neu ausgewertet. Die Wandlung gilt als abgeschlossen, wenn die Referenzspannung die Messspannung über- oder unterschritten hat. In diesem Fall ist sichergestellt, dass sich die zu messende Spannung zwischen den letzten beiden Referenzspannungen befindet. Da der Tracking-Wandler im schlechtesten Fall 2^n Schritte benötigt, eignet sich dieser nicht zur Messung von großen Spannungsänderungen. Spannungsverläufe, die einer nur geringen Veränderungsrate unterliegen, können hingegen effizient mit dem Tracking-Wandler digitalisiert werden.

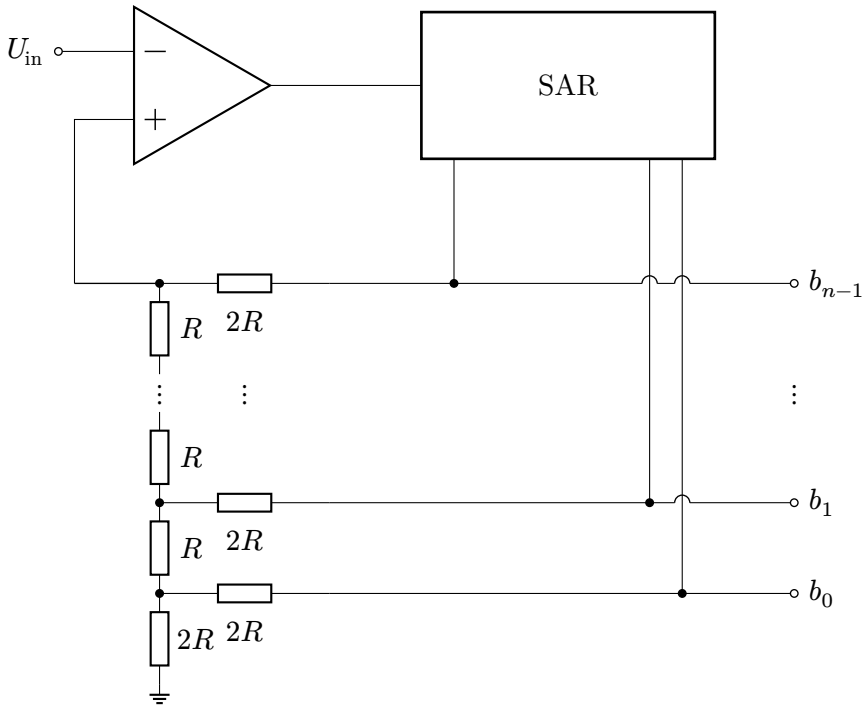


Abbildung 1.6: Sukzessive-Approximation-Wandler

Sukzessive-Approximation-Wandler (SAR)

Eine weitere Möglichkeit zur Realisierung einer Analog-Digital-Wandlung ist die sukzessive Approximation. Dieses Verfahren arbeitet nach dem Prinzip der Binärsuche. Ein Sukzessive-Approximation-Register (SAR, Abbildung 1.6) wird verwendet. Die Funktionsweise des SAR zeigt Abbildung 1.7 für einen 3-Bit-Wandler. Die Referenzspannung wird, wie beim Tracking-Wandler, in einer n-Bit Variable r festgehalten. Zu Beginn einer Wandlung wird die Referenzspannung auf 0 gesetzt. Im ersten Schritt wird testweise das „Most Significant Bit“ (MSB) der Variable r auf 1 gesetzt. Anschließend wird der Wert von r als Referenzspannung mit Hilfe des DA-Wandlers ausgegeben. Wie beim Tracking-Wandler vergleicht ein Komparator die Mess- und Referenzspannung. Ist die Messspannung größer als die Referenzspannung (Komparator $K = 0$), wird die 1 beibehalten, andernfalls ($K = 1$) wird das Bit auf 0 zurückgesetzt. Danach wird analog mit dem nächstniedrigerwertigem Bit verfahren. Die Wandlung ist beendet, wenn alle Bits überprüft wurden. Der nach Überprüfung aller Bits in r gespeicherte Wert ist das Ergebnis der Wandlung. Für eine komplette Wandlung werden somit stets n Schritte benötigt.

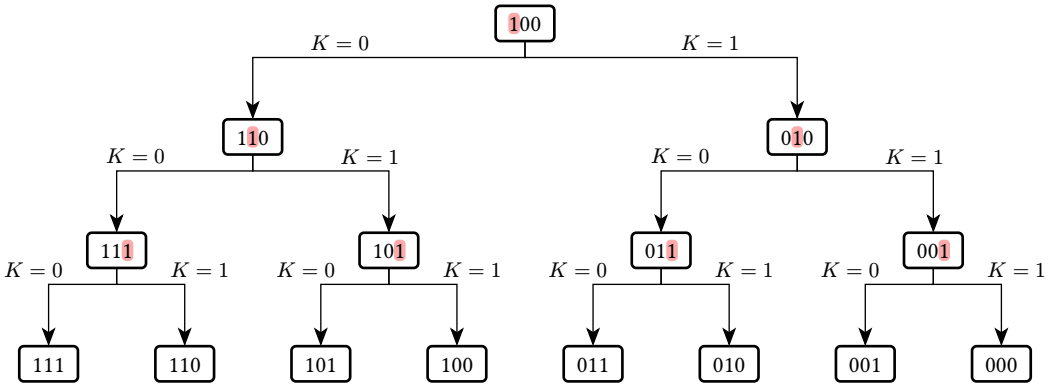


Abbildung 1.7: SAR Entscheidungsbaum für $n = 3$
(K ist jeweils das Ergebnis des Komparators)

? Lernerfolgsfragen

- Warum wird bei der sukzessiven Approximation das MSB zuerst testweise gesetzt?
- Benötigt ein Sukzessive-Approximation-Wandler immer weniger Schritte als ein Tracking-Wandler?

1.2 Hausaufgaben

Hinweise zu den Hausaufgaben

Lösen Sie alle hier vorgestellten Aufgaben und geben Ihre Lösungen fristgerecht im Moodle-Lernraum ab. Ihre Abgabe soll dabei enthalten:

- Alle .c/.h-Dateien
- Alle Projektdateien: .atsln, .xml und .cproj
- `Makefile`

Es sollen keine Testtasks, PDFs oder Doxygen-Dateien abgegeben werden.

> **! Achtung**
>
> Alle im Versuch geforderten Funktionalitäten müssen implementiert sein!

> **! Achtung**
>
> Ihr Code muss mit Testtasks und aktivierten Compiler-Optimierungen ohne Fehler und ohne Warnungen kompilieren. Wie Sie die Optimierungen einschalten ist dem Kapitel *6.3.2* im *Begleitenden Dokument* zu entnehmen.

Verwenden Sie zur Prüfung auf Warnungen den Befehl „Rebuild Solution“ im „Build“-Menü von Microchip Studio 7. Die übrigen in der grafischen Oberfläche angezeigten Buttons führen nur ein inkrementelles Kompilieren aus, d.h. es werden nur geänderte Dateien neu kompiliert. Warnungen und Fehlermeldungen in unveränderten Dateien werden dabei nicht ausgegeben.

Nutzung des Testpools

Da dieser Versuch mit mehreren Hardwarekonfigurationen arbeitet, wurden die PCs im Testpool in die Kategorien *V1: AD/DA* und *V1: ADC Menü* unterteilt.
Der Programmcode für die Aufgaben zur AD/DA-Wandlung kann auf VMs der ersten Kategorie getestet werden. Steuerelemente in der Software auf den VMs ermöglichen das Erzeugen einer Messspannung sowie das Bedienen der DIP-Schalter.

> **Hinweis**
>
> Aufgrund von Ungenauigkeiten bei der Spannungserzeugung und -messung kann es zu Abweichungen von $(\pm 0{,}2)\,\mathrm{V}$ kommen.

Analog dazu kann Programmcode für die Aufgaben zum ATmega644 auf VMs der zweiten Kategorie getestet werden.

Hinweis

Pin A0 ist für die Messung der Spannung des internen AD-Wandlers reserviert und kann daher nicht für die Binäruhr verwendet werden.

1.2.1 Allgemeines

Folgende Dokumente, die im Moodle-Lernraum des Praktikums verfügbar sind, dienen dem grundlegenden Verständnis:

- **Bedienungsanweisung:** Dieses Dokument stellt Ihnen die sichere Bedienung des Multimeters und der Spannungsquelle vor, die Sie für die Präsenzaufgaben benötigen.
- **Begleitende Dokument:** Dieses Dokument enthält alle Informationen, die Sie für den Einstieg in die C-Programmierung eines Mikrocontrollers benötigen. Es dient als grundlegendes Nachschlagewerk und beinhaltet eine Beschreibung aller für dieses Praktikum benötigten Informationen.
- **Versuch 0:** Dieser optionale Versuch dient als sanfter Einstieg in die Mikrocontrollerprogrammierung und vermittelt grundlegende Kenntnisse in der Programmiersprache C.

In diesem Versuch werden zum Einstieg in die Mikrocontrollerprogrammierung mehrere elementare Applikationen implementiert.
Zuerst wird eine für Mikrocontroller typische Anwendung, die Analog-Digital- und Digital-Analog-Wandlung (AD/DA-Wandlung), implementiert. Dabei wird neben dem Evaluationsboard eine weitere Platine (siehe Abbildung 1.2) verwendet, die die für die AD/DA-Wandlung benötigte Hardware bereitstellt.
Im zweiten Teil des Versuchs werden weitere Komponenten des ATmega644 vorgestellt und relevante Konzepte der Mikrocontrollerprogrammierung vertieft. Dazu wird eine Applikation mit mehreren einfachen Unterprogrammen implementiert. Hierzu zählt beispielsweise eine Binäruhr und die Ausgabe von verschiedenen Informationen auf einem LED-Bargraph.

1.2.2 Aufgaben zur AD/DA-Wandlung

In diesem Teil des Versuchs sollen Sie die im Grundlagenkapitel vorgestellten Wandler implementieren. Beachten Sie, dass für diesen Teil des Versuchs eine AD/DA-Platine auf das Evaluationsboard aufgesetzt wird (siehe Abbildung 1.2). Die Pinbelegung für diesen Aufgabenteil können Sie Pinbelegung 1.1 am Ende des Versuchsdokuments entnehmen. Das LC-Display wird für diesen Teil des Versuchs nicht verwendet.

Entwickeln Sie basierend auf dem bereitgestellten Codegerüst für den ATmega644 ein Programm, das folgende Funktionalitäten enthält:

1. Eine manuelle Ansteuerung des R-2R-Netzwerks
2. Einen Sukzessive-Approximation-Wandler
3. Einen Tracking-Wandler

Fügen Sie für jede der oben aufgezählten Funktionalitäten eine Funktion hinzu. Die manuelle Ansteuerung kann beispielsweise als void manuell(void) realisiert werden. Fügen Sie neben den drei zu implementierenden Funktionen eine int main(void)-Funktion hinzu. In dieser soll je nach Bedarf eines der drei Unterprogramme ausgewählt werden. Eine switch-Anweisung eignet sich zur Selektierung des gewünschten Unterprogramms, die eine zuvor deklarierte Variable auswertet und entsprechend ihres Wertes eines der Unterprogramme startet. Es genügt, das zu startende Unterprogramm im Code festlegen zu können. Selbstverständlich steht es Ihnen frei, beliebige weitere Hilfsprogramme anzulegen.

DA-Wandlung: Ansteuerung des R-2R-Netzwerks

Beginnen Sie mit der Funktion zur manuellen Ansteuerung der AD/DA-Platine. In dieser Funktion soll der durch den DIP-Schalter (Markierung 1 in Abbildung 1.2) eingestellte Bit-Wert intern im Mikrocontroller auf das R-2R-Netzwerk abgebildet werden. Legen Sie diesen Wert ebenfalls an die LEDs an, um zu überprüfen, welche Bitkombination aktiv ist. Hierzu konfigurieren Sie zunächst Port D (DIP-Schalter) als Eingang. Entsprechend müssen Port A (LEDs) und Port B (R-2R-Netzwerk) als Ausgang konfiguriert werden. Lesen Sie anschließend den Wert in PIND und schreiben Sie diesen geeignet in PORTA und PORTB. Beachten Sie, dass im entsprechenden Bit in PIND eine 0 steht, wenn der zugehörige DIP-Schalter auf ON geschaltet ist.
Achten Sie auf die korrekte Initialisierung aller relevanten Register. Beachten Sie, dass Änderungen am DIP-Schalter zur Laufzeit direkt umgesetzt werden sollen. Der Mikrocontroller soll den neu eingestellten Wert kontinuierlich am R-2R-Netzwerk und an die LEDs ausgeben, ohne dass ein Neustart des Mikrocontrollers nötig ist.

AD-Wandlung: Tracking-Wandler und SA-Wandler

Implementieren Sie als Nächstes den Tracking-Wandler und den SA-Wandler. Beide wurden im Grundlagenkapitel vorgestellt und sollen gemäß dieser Beschreibung programmiert werden. Verwenden Sie zur Erzeugung der Referenzspannung das R-2R-Netzwerk, dessen Ausgang mit dem Komparator verbunden ist. Die zu messende Spannung wird von einer externen Spannungsquelle erzeugt, die an der AD/DA-Platine angeschlossen wird. Für den auf der AD/DA-Platine befindlichen Komparator gilt folgendes Verhalten:

- $U_{\text{ref}} < U_{\text{mess}} \Rightarrow \text{COMP_OUT} = 0$
- $U_{\text{ref}} > U_{\text{mess}} \Rightarrow \text{COMP_OUT} = 1$

Hinweis

Aufgrund der hohen Ausführungsgeschwindigkeit des Mikrocontrollers ist es notwendig, nach jeder Änderung am R-2R-Netzwerk etwas zu warten, bis das Ergebnis des Komparators ausgelesen werden kann. Wählen Sie eine Wartezeit von mindestens 50 ms.

Benutzen Sie zum Warten die Delay-Funktionen aus der Bibliothek `util/delay.h`, die mit einem entsprechenden `#include`-Befehl in den Code eingebunden werden muss. Informationen über das Einbinden von Bibliotheken können dem Kapitel *5.1.2* im *Begleitenden Dokument* entnommen werden. Die eingebundene Headerdatei stellt folgende Funktionen zur Verfügung:

- `void _delay_us(double __us)`
- `void _delay_ms(double __ms)`

Geben Sie während der Wandlung stets die aktuelle Referenzspannung auf die an Port A angeschlossenen LEDs aus, damit der Wandlungsvorgang visuell nachvollzogen werden kann.

Implementieren Sie beide Wandler so, dass eine vollständige Wandlung der aktuell angelegten Spannung erst dann erfolgt, wenn der Button an Pin C1 gedrückt wurde. Der Button befindet sich auf der AD/DA-Platine. Nachdem eine komplette Wandlung durchgeführt wurde, soll das Programm erst bei erneutem Tastendruck die nächste Wandlung durchführen.
Da in diesem und kommenden Versuchen häufig mit Buttons gearbeitet wird, bietet es sich an, ein universell einsetzbares Modul zur Benutzung zu implementieren. Ihnen werden für diese Aufgabe die Dateien `os_input.c` und `os_input.h` vorgegeben, die die Struktur dieses Moduls beinhalten. Die Funktion `void os_initInput(void)` dient zur Konfiguration des entsprechenden Ports, mit dem der Button verbunden werden soll. Verwenden Sie die Funktion `uint8_t os_getInput(void)` zur Zustandsabfrage der Buttons. Achten Sie hierbei besonders auf die Erklärung im Funktionskommentar, der verschiedene Szenarien für den Rückgabewert beschreibt. Implementieren Sie die Funktionen `void os_waitForInput(void)` und `void os_waitForNoInput(void)`. Verwenden Sie diese in Ihrem Projekt dort, wo auf das Drücken bzw. Loslassen des Buttons gewartet werden soll.

1.2.3 Aufgaben zum ATmega644

In diesem Hausaufgabenteil sollen weiterführende Funktionen für den ATmega644 implementiert werden. Hierzu wird ein separates Projekt verwendet, das Ihnen ebenfalls als eigenes Codegerüst zur Verfügung gestellt wird. Im Unterschied zum ersten Teil des Versuchs arbeiten Sie hier nur mit dem Evaluationsboard ohne die aufgesetzte AD/DA-Platine. In Pinbelegung 1.2 finden Sie die für diesen Versuchsteil geltende Pinbelegung. Die in diesem Abschnitt erläuterten Aufgaben dienen dazu, weitere fundamentale Konzepte der Mikrocontrollerprogrammierung zu verstehen und mit deren Besonderheiten umzugehen. Im vorgegebenen Codegerüst ist bereits eine einfache Menüführung

implementiert, über die mehrere Unterprogramme ausgeführt werden können. Diese Unterprogramme sind jedoch noch nicht vollständig implementiert und müssen von Ihnen ergänzt werden.

In der main-Funktion des Projekts wird zuerst das Buttonmodul und der Treiber für das LC-Display, das in dieser Aufgabe verwendet wird, initialisiert. Nach der Initialisierung wird das Menü auf dem LCD angezeigt. Die Implementierung des Menüs befindet sich in der Datei menu.c und wird durch die Funktion void showMenu(void) realisiert. Nach der Auswahl eines Unterprogramms im Menü wird die Funktion void start(uint8_t programIndex) verwendet, um in die entsprechenden Unterprogramme zu verzweigen. Darüber hinaus initialisiert diese Funktion die für das entsprechende Unterprogramm nötigen Module. Die Unterprogramme sind so konzipiert, dass ein Zurückkehren in das Hauptmenü möglich ist.

Das Buttonmodul

Die erste Aufgabe besteht darin, das Buttonmodul zu implementieren, das zur Bedienung des Menüs verwendet wird. An den Pins C0, C1, C6 und C7 sind nun Buttons angeschlossen, die als Enter, Down, Up und ESC bezeichnet werden. Vervollständigen Sie in der Datei os_input.c die Funktionen void os_initInput(void), uint8_t os_getInput(void), void os_waitForInput(void) und void os_waitForNoInput(void). Es steht Ihnen frei, die aus dem ersten Versuchsteil vorhandene Implementierung dieser Funktionen zu übernehmen und anzupassen. Beachten Sie jedoch die für diesen Versuchsteil veränderte Anzahl und Verkabelung der Buttons.

Hinweis

Die unteren vier Bits des Rückgabewertes der Funktion uint8_t os_getInput(void) sollen, entsprechend dem Kommentar im Codegerüst, den Status der Buttons repräsentieren.

Programm 1: Hello World

Als erstes Unterprogramm soll die Funktion void helloWorld(void) vervollständigt werden. Dieser Teil des Versuchs stellt eine Einführung in die Verwendung des LCDs dar. Der LCD-Treiber wird bereits an mehreren Stellen im vorgegebenen Codegerüst verwendet (siehe Funktion void showMenu(void)). Sie können diese als zusätzliche Anwendungsbeispiele heranziehen. Der Ablauf der Funktion void helloWorld(void) soll wie folgt sein:

1. Mit Hilfe des LCD-Treibers die Zeichenkette „Hallo Welt!“ auf dem LCD ausgeben
2. 500 ms warten
3. Die Zeichenkette wieder von dem LCD entfernen
4. Weitere 500 ms warten

5. Die Schritte 1. bis 4. solange wiederholen, bis ESC gedrückt wird.

Sobald ESC gedrückt und losgelassen wurde, soll ins Hauptmenü zurückgekehrt werden. Verwenden Sie dazu Ihr Buttonmodul. Die Verwendung eines Interrupts ist nicht erforderlich.

Programm 2: Binäruhr

Das zweite Unterprogramm soll eine Binäruhr im 12-Stunden-Format realisieren. Die Binäruhr zeigt eine im Binärformat codierte Uhrzeit mit Hilfe der auf dem Evaluationsboard befindlichen LEDs an. Zusätzlich wird die Uhrzeit auf dem LCD ausgegeben. In dieser Implementierung werden Stunden, Minuten und Sekunden auf der Binäruhr angezeigt. Die vorhandenen LEDs werden entsprechend in drei Bereiche unterteilt, die mit diesen drei Größen assoziiert werden. Wird nun für jede Größe der Leuchtzustand der entsprechenden LEDs als Binärzahl interpretiert, so lässt sich daraus die aktuelle Uhrzeit ableiten. Da zur Ausgabe der Binäruhr nicht mehr als 16 LEDs zur Verfügung stehen, wird die Uhrzeit im 12-Stunden-Format verwaltet, wie auf einer analogen Uhr. Anders als im 24-Stunden-Format gibt es keine Uhrzeiten 0:00 bis 0:59; stattdessen werden 12:00 bis 12:59 verwendet. Die Zeitzählung fängt dementsprechend bei 12:00 an. Um 12:59 springt die Uhr direkt auf 1:00 und wird dann wie gewohnt wieder bis 12:59 hochgezählt.

Es werden jeweils sechs LEDs für den Wert der Sekunden und Minuten sowie vier LEDs für den Wert der Stunden verwendet. In Abbildung 1.8 ist exemplarisch die Ausgabe der Uhrzeit 5:26:53 auf den LEDs gezeigt.
Zusätzlich zur Ausgabe auf den LEDs soll die Uhrzeit in der üblichen Dezimaldarstellung auf dem LCD ausgegeben werden. Da auf dem LCD mehr Platz zur Verfügung steht, werden hier zusätzlich Millisekunden angezeigt. Der Ablauf dieses Unterprogramms kann Abbildung 1.9 entnommen werden.

Zur Realisierung der Binäruhr dienen neben der Anzeigefunktion void displayClock(void) und die Dateien bin_clock.c und led.c. Im Folgenden wird schrittweise die Vervollständigung von bin_clock.c erläutert. Legen Sie als Erstes vier Variablen für Stunden, Minuten, Sekunden und Millisekunden in diesem Modul an, die so initialisiert werden sollen, dass die Uhr bei 12:59:45:000 startet. Beachten Sie die korrekte Dimensionierung der Variablen. Lesen Sie im Kapitel *5.1.4 Datentypen* im *Begleitenden Dokument* nach, welche Datentypen für den Wertebereich der Variablen geeignet sind.

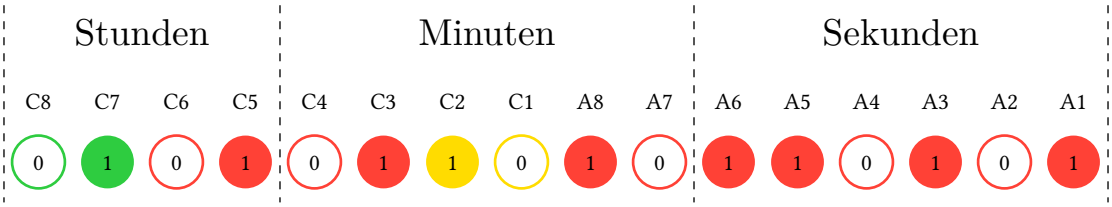


Abbildung 1.8: Ausgabe der Uhrzeit 5:26:53 auf den LEDs

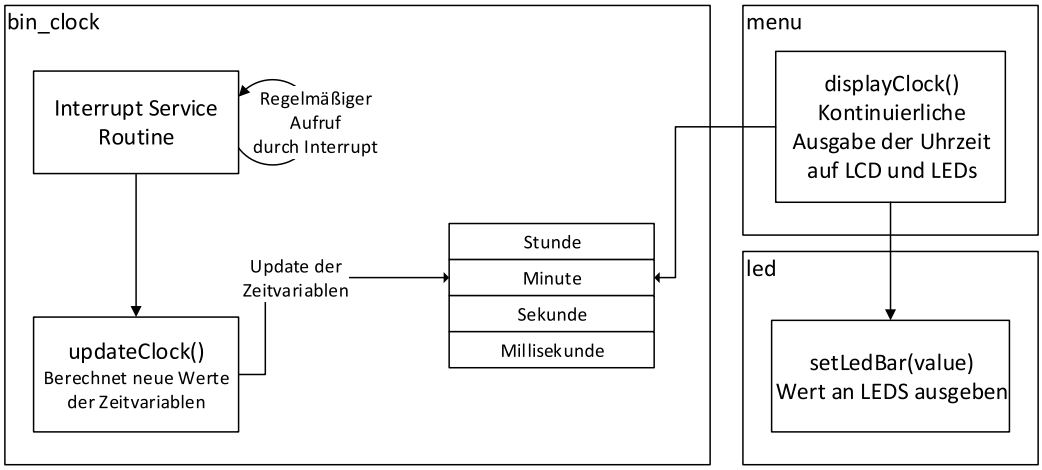


Abbildung 1.9: Architektur der Binäruhr

Zur Implementierung der Uhr wird ein interner Timer des ATmega644 verwendet. Der Timer aktiviert in gleichen Abständen einen Interrupt. Bei jedem Aufruf des Interrupts müssen die entsprechenden Zeitvariablen aktualisiert werden.
Die Ihnen vorgegebene Initialisierungsfunktion void initClock(void) dient dazu, einen Timer des ATmega644 so zu initialisieren, dass dieser in einem Intervall von 10 ms einen Interrupt aktiviert. Hierzu wurde der Timer im *Clear Timer on Compare Match* (CTC) Modus konfiguriert. Der im TCCR0B-Register hinterlegte Prescaler von 1024 bewirkt, dass das Zählregister TCNT0 bei jedem 1024. Takt des Oszillators um Eins inkrementiert wird. Bei einem Prozessortakt von 20 MHz wird das Zählregister somit alle $\frac{1}{20\,000\,000} \cdot 1\,024\,\text{s} = 51{,}2\,\mu\text{s}$ um 1 inkrementiert. Aufgrund des CTC-Modus löst der Timer genau dann einen Tick aus, wenn das Zählregister den im OCR0A-Register hinterlegten Wert (hier 195) erreicht hat. Somit ergibt sich ein Intervall von $51{,}2\,\mu\text{s} \cdot 195 \approx 10\,\text{ms}$. Beachten Sie, dass ein Aufruf der initClock-Funktion im vorgegebenen Codegerüst bereits existiert. Nachdem der Timer konfiguriert und aktiviert wurde, wird im Takt von 10 ms die Interrupt-Service-Routine ISR(TIMER0_COMPA_vect) aufgerufen. In dieser ist der Wert der Millisekunden-Variable entsprechend des Timer-Intervalls anzupassen.
Rufen Sie nach dem Anpassen der Millisekunden-Variable die Funktion void updateClock(void) auf. Diese soll die Werte der übrigen Zeitvariablen korrekt anpassen. Erreicht der Wert der Millisekunden-Variable beispielsweise den Wert 1000, so muss die Sekunden-Variable inkrementiert und die Millisekunden-Variable auf 0 zurückgesetzt werden.

Die zweite benötigte Komponente für die Binäruhr ist die Datei led.c. In dieser soll eine Ansteuerung des LED-Bargraphen des Mikrocontrollers realisiert werden.

Wert	8	4	2	1	32	16	8	4	2	1	32	16	8	4	2	1
Format	H	H	H	H	M	M	M	M	M	M	S	S	S	S	S	S
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Pin	D7	D6	D5	D4	D3	D2	D1	D0	A7	A6	A5	A4	A3	A2	A1	A0

Tabelle 1.3: Ausgabeformat der Binäruhr auf dem LED-Bargraph

Bedingt durch die Verschaltung der LEDs leuchten diese genau dann, wenn der Ausgang des Mikrocontrollers einen Low-Pegel (0 V) aufweist. Die Funktion `void setLedBar(uint16_t value)` soll den übergebenen 16-Bit Parameter so auf den LED-Bargraphen umsetzen, dass eine LED genau dann leuchtet, wenn das entsprechende Bit im übergebenen Parameter 1 ist. Für diese Transformation eignet sich der Operator `~`, der das bitweise Invertieren einer Variable ermöglicht (siehe Kapitel *5.1.13: Operatoren* im *Begleitenden Dokument*). Das *Least Significant Bit* (LSB) des Parameters soll auf Pin `A0` und das MSB auf Pin `D7` ausgegeben werden (siehe Tabelle 1.3). Der Parameter `activateLedMask` ist für dieses Unterprogramm nicht relevant und muss daher nicht berücksichtigt werden. Beachten Sie hierbei, dass ein Aufruf der Funktion `void initLedBar(void)` bereits im Codegerüst vorhanden ist.

Nach Implementierung der Binäruhr- und LED-Funktionen können diese von Ihnen verwendet werden, um die vorgegebene Funktion `void displayClock(void)` zu vervollständigen. Legen Sie eine temporäre Variable `uint16_t clockVal` an und verwenden Sie Bitshift-Operationen, um die Zeitvariablen an die entsprechenden Positionen in diese Variable zu verschieben. Rufen Sie anschließend `setLedBar(clockVal)`, um die Uhrzeit auf den LEDs auszugeben. Bei der Auswahl des entsprechenden Menüpunkts im vorgegebenen Menüdialog wird diese Funktion aufgerufen. Analog zu dem ersten Unterprogramm soll auch dieses Unterprogramm solange aktiv bleiben, bis `ESC` gedrückt wird. Solange dies nicht geschehen ist, soll die Uhrzeit auf dem Display des Mikrocontrollers und dem LED-Bargraphen ausgegeben werden. Verwenden Sie hierbei die Getter-Funktionen aus `bin_clock.c`. Auf dem Display und den LEDs sollen permanent die aktuellen Größen für Stunden, Minuten, Sekunden und Millisekunden angezeigt werden. Zur besseren Lesbarkeit der Uhrzeit auf dem Display sollen die ausgegebenen Zahlen in das Format „HH:MM:SS:mmm“ transformiert werden. Hierzu muss der auszugebende Zahlenwert auf seine Größe analysiert werden, bevor er auf dem Display ausgegeben wird. Weist dieser zu wenige Stellen für das gewünschte Format auf, so müssen eine oder mehrere Nullen durch eine entsprechende Ausgabe auf dem Display vorangestellt werden. Beim Verlassen des Unterprogramms durch `ESC` muss der Timer nicht ausgeschaltet werden.

Hinweis

Achten Sie darauf, mögliches Flackern des LCDs zu minimieren, indem Sie Aufrufe von void lcd_clear(void) und erneutes Schreiben von unveränderten Zeichen und Zahlen vermeiden.

Programm 3: Interner AD-Wandler

Sie haben bereits im ersten Teil des Versuchs grundlegende Techniken zur AD-Wandlung kennengelernt. Dabei haben Sie die AD/DA-Platine verwendet und die entsprechenden Wandler selbst implementiert. Der ATmega644 verfügt jedoch auch über einen integrierten AD-Wandler. Dieser wendet das Prinzip eines SA-Wandlers an und besitzt eine Auflösung von 10 Bit. Seine Geschwindigkeit wird abhängig vom Prozessortakt gesteuert und sollte in dem Frequenzband von $(50 - 200)\,\mathrm{kHz}$ liegen. Die Wandlungsfrequenz des integrierten AD-Wandlers kann mit Hilfe des Registers ADCSRA angepasst werden, der einen Prescaler zum Prozessortakt festlegt. Um innerhalb des gültigen Frequenzbandes zu bleiben, ist ein Prescaler von 128 gewählt worden. Die Frequenz des AD-Wandlers beträgt somit $\frac{20\,\mathrm{MHz}}{128} \approx 156\,\mathrm{kHz}$. Als Eingang für die zu messende Spannung können verschiedene Pins ausgewählt werden. Diese werden beim ATmega644 als ADC0 bis ADC7 bezeichnet und sind mit den Pins A0 bis A7 verbunden. Für diese Aufgabe wurde der Kanal ADC0 gewählt, der mit Pin A0 verbunden ist. Beachten Sie daher, dass Ihnen der Pin A0 nicht für die entsprechende LED zur Verfügung steht (siehe Pinbelegung 1.2).
Die Funktion void initAdc(void) dient der Initialisierung der soeben erläuterten Register und ist Ihnen vollständig vorgegeben. Beachten Sie, dass ein Aufruf dieser Funktion bereits im Codegerüst vorhanden ist. Das Ergebnis einer Wandlung wird von der ebenfalls vorgegebenen Funktion uint16_t getAdcValue(void) zurückgeliefert. Implementieren Sie die Funktion void displayAdc(void), die folgende Aufgaben erfüllen soll:

- Anzeigen der Spannung in der ersten Zeile des Displays
- Ausgabe der Spannung auf dem LED-Bargraphen (Format siehe Tabelle 1.4)
- Ein Untermenü in der zweiten Zeile des Displays, die das Speichern und Anzeigen von Spannungswerten ermöglicht

Anzeige der Spannung

Es bietet sich an, das gesamte Programm in Form einer Endlosschleife aufzubauen, die eine Verzögerung von 100 ms enthält und in jedem Schleifendurchlauf die Spannung misst und ausgibt. Somit wird eine sowohl flüssige als auch flackerfreie Darstellung der Informationen gewährleistet. Analog zu den Programmen 1 und 2 soll auch dieses durch das Drücken von ESC verlassen werden.

Abbildung 1.10 zeigt das ADC Menü exemplarisch. Es soll kontinuierlich die zuletzt gemessene Spannung sowohl auf dem LCD als auch auf dem LED-Bargraphen ange-

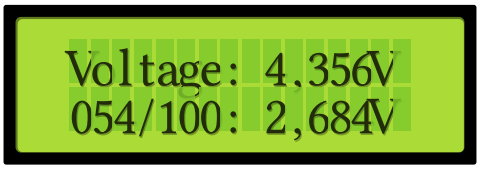


Abbildung 1.10: Beispielhafte Ausgabe des ADC-Menüs

Spannung (V)	5,0	4,7	4,3	4,0	3,7	3,3	3,0	2,7	2,3	2,0	1,7	1,3	1,0	0,7	0,3	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Pin	D7	D6	D5	D4	D3	D2	D1	D0	A7	A6	A5	A4	A3	A2	A1	A0

Tabelle 1.4: Ausgabe der digitalisierten Spannung auf dem LED-Bargraph

zeigt werden. Zum Anzeigen der Spannung auf dem Display eignet sich die Funktion void lcd_writeVoltage(...) des LCD-Treibers.
Die Ausgabe der Spannung auf dem LED-Bargraphen soll, wie in Tabelle 1.4 gezeigt, linear erfolgen. Es stehen 15 LEDs zur Verfügung und die obere Schranke des zu messenden diskreten Spannungswertes liegt bei 1023. Somit beträgt die Auflösung der Ausgabe pro LED $\frac{5\,\mathrm{V}}{15} = 0{,}3\,\mathrm{V}$ und bezogen auf die diskret hinterlegte Spannung $\frac{1023}{15} = 68$. In Abbildung 1.11 sind beispielhaft drei verschiedene Spannungsausgaben auf dem LED-Bargraphen dargestellt.

Für die Umwandlung des Rückgabewertes der Funktion uint16_t getAdcValue(void) in eine Binärzahl, die zur Ausgabe auf dem LED-Bargraphen geeignet ist, bietet sich folgendes Vorgehen an:

1. Initialisierung einer 16-Bit Variable ledValue mit 0. Jedes Bit dieser Variable ist stellvertretend für eine LED auf den Bargraphen. Es stehen jedoch nur 15 LEDs zur Verfügung, da Pin A0 für die AD-Wandlung verwendet wird, weshalb die dazu zugehörige LED nicht verwendet werden kann.

2. Initialisierung einer 16-Bit Variable adcResult mit dem Rückgabewert der Funktion getAdcValue.

3. Wie oben erläutert repräsentiert der Wert 68 der Variable adcResult eine Spannung von 0,3 V auf den LED-Bargraphen. Damit der Bargraph die gesamte Spannung widerspiegelt, müssen die folgenden Schritte wiederholt werden, bis adcResult kleiner als 68 ist:

   a) ledValue um 1 nach links shiften (entspricht der Multiplikation mit 2).

   b) ledValue mit dem Wert 2 verodern (entspricht der Addition von 2). Dieses Vorgehen bewirkt, dass der Bargraph entsprechend der Spannung gefüllt

wird, wobei Pin A0 unberührt bleibt, da dieser nicht für den Bargraphen verwendet werden darf.

c) Von der Variablen adcResult den Wert 68 subtrahieren.

Im Anschluss an die Umformung kann die Variable ledValue auf dem LED-Bargraphen ausgegeben werden. Verwenden Sie hierfür die bereits angelegte und im vorhergehenden Aufgabenteil verwendete Datei led.c. Die Datei muss zuvor jedoch so erweitert werden, dass nur bestimmte LEDs zur Ausgabe genutzt werden, da Pin A0 für die AD-Wandlung verwendet wird. Wird dennoch auf diesen Pin zugegriffen, so resultiert dies in einer fehlerhaften Spannungsmessung. Hierzu dient der Parameter activateLedMask aus der Datei led.c, der eine Bitmaske für aktivierte LEDs darstellt. Es sollen nur diejenigen LEDs ansteuerbar sein, die mit einer 1 in der Bitmaske gekennzeichnet sind. Passen Sie die Funktion void initLedBar(void) sowie void setLedBar(uint16_t value) so an, dass nur diejenigen Bits der DDR- und PORT-Register verändert werden, die in der Variable activateLedMask mit einer 1 markiert sind. Die Zuordnung der Bits dieser Variable zu den Pins des Mikrocontrollers geschieht analog zu dem Parameter der Funktion setLedBar (siehe auch Tabelle 1.4). Beachten Sie hierbei, dass ein Aufruf der Funktion void initLedBar(void) und das Setzen der Variable activateLedMask mit einer geeigneten Bitmaske für jedes Unterprogramm bereits im Codegerüst vorhanden ist.

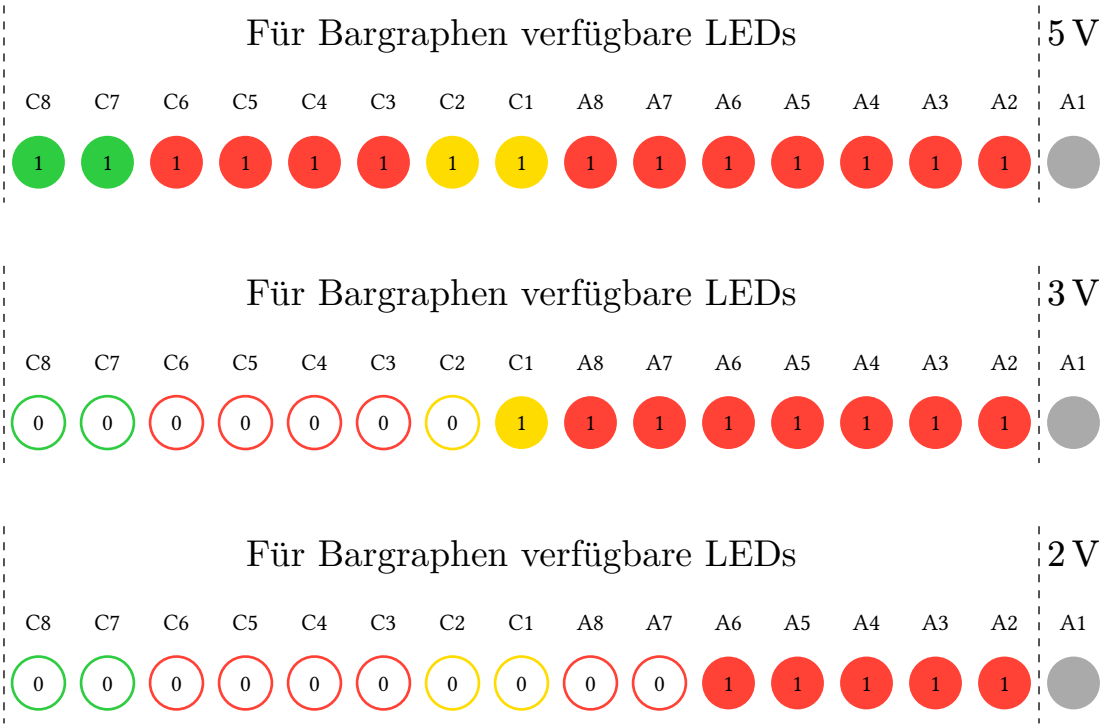


Abbildung 1.11: Bargraph bei Messung von 5 V (oben), 3 V (mitte) und 2 V (unten)

Speichern von Messwerten

Neben der Anzeige der momentan gemessenen Spannung soll es möglich sein, Messwerte zu speichern und diese zu einem späteren Zeitpunkt anzuzeigen. Zur Benutzerinteraktion muss die Funktion void displayAdc(void) um eine Menüführung erweitert werden. Messwerte sollen abgespeichert werden, wenn Enter gedrückt wird. Um auch mehrere Messwerte nacheinander speichern zu können, soll nicht auf das Loslassen des Buttons gewartet werden. Integrieren Sie daher das Speichern von Messwerten in die Hauptschleife des Programms, um in einem Intervall von 100 ms Messwerte aufzeichnen zu können.
Mithilfe von Up und Down soll es zudem möglich sein, die Spannungswerte aus dem Puffer auszulesen und in der zweiten Zeile des LCD gemäß Abbildung 1.10 anzuzeigen. Verwenden Sie hierzu einen lokalen Zählindex in der Funktion void displayAdc(void). Die Anzeige der Spannungswerte soll durch die separate Funktion displayVoltageBuffer realisiert werden, die als Parameter den Index des anzuzeigenden Spannungswertes erhält. Wurde einer der Buttons gedrückt, soll nicht auf das Loslassen dieses gewartet werden, sondern der Anzeigeindex im Intervall von 100 ms inkrementiert bzw. dekrementiert werden, was das *Blättern* durch die Messwerte ermöglicht.
Eine automatische Inkrementierung des Anzeigeindexes beim Speichern von neuen Werten ist nicht erforderlich. Wichtig ist hingegen, dass die angezeigten Indizes innerhalb der Grenzen des Puffers liegen. Achten Sie bei der Ausgabe der Indizes darauf, bei Bedarf führende Nullen einzufügen, um eine gute Lesbarkeit beim Blättern zu gewährleisten.

Implementieren Sie die Datenbank zur Speicherung der Messwerte in der Datei adc.c. Zur Initialisierung und Verwendung dieser Datenbank werden Ihnen Funktionsrümpfe bereitgestellt, die vervollständigt werden müssen. Die Datenbank soll als Puffer realisiert werden, auf den mit Zeigern zugegriffen werden kann. Soll ein neuer Messwert gespeichert werden, wird die Funktion storeVoltage des ADC-Moduls aufgerufen und der zuletzt gemessene Spannungswert unter dem ersten freien Index des Puffers gespeichert. Anschließend soll der Pufferindex der Datenbank unter Beachtung der Größe des Puffers um eine Position verschoben werden. Der Wertebereich von diesem Index reicht von 0 bis zum Rückgabewert der Funktion getBufferSize() − 1. Ist die maximale Indexposition erreicht, sollen keine weiteren Messwerte gesichert und der Index nicht weiter inkrementiert werden.

Implementierung der Datenbank

Die Funktion storeVoltage dient gleichermaßen zum Speichern von Messwerten, als auch zur Initialisierung der Datenbank. Ist beim Aufruf der Funktion die Puffergröße gleich 0, muss zuerst Speicher für den Puffer alloziert werden. Dies wird mit Hilfe der Funktion malloc durchgeführt, die durch die Standard C-Bibliothek zur Verfügung gestellt wird. Diese Bibliothek kann durch die Headerdatei stdlib.h verwendet werden. Als Argument erhält die malloc-Funktion die gewünschte Größe des Speicherbereichs in Bytes. Der Rückgabewert ist die Adresse des ersten Bytes des Speicherbereichs. Hat der Rückgabewert die Adresse 0, steht nicht genügend Speicherplatz zur Verfügung und der Puffer

```c
uint16_t *position = malloc(10 * sizeof(uint16_t)); // malloc -> 270/271
```
Ein Zeiger wird angelegt, der im weiteren Verlauf auf das erste Byte des Speicherbereichs zeigen soll. Als Rückgabewert der Funktion malloc wird in diesem Beispiel ein Zeiger auf die 2-Byte-Speicherstelle von Adresse 270 bis 271 angenommen.

```c
uint8_t offset = 4;

*position = 155;
```
Mit Hilfe des Operators * kann bei Zeigern auf den Inhalt an einer Adresse zugegriffen werden. Hier wird der Inhalt an Adresse 270/271 auf den Wert 155 verändert.

```c

*(position + offset) = 1000;
```
Die Wahl eines uint16_t * Zeigers bewirkt, dass das Offset nicht auf einzelne 1-Byte-Speicherstellen sondern auf 2-Byte-Speicherstellen bezogen wird. Wäre position vom Typ uint8_t *, so würde dieser Zugriff auf die Speicherstelle 274 statt 278/279 erfolgen.

```c

uint16_t value = *(position + offset); // value = 1000
```
Analog zum Schreibvorgang wird hier der Lesevorgang mit Offset durchgeführt.

```c

value = position[offset]; // value = 1000
```
Diese Schreibweise ist bei Zeigern äquivalent zu der vorherigen und führt zum gleichen Ergebnis.

Listing 1.1: Verwendung von Zeigern

kann nicht angelegt werden. Für diese Aufgabe soll ein Puffer für 100 Spannungswerte angelegt werden. Für einen Spannungswert müssen im Puffer 2 Bytes Speicher reserviert werden. Intern wird der nächste freie Index im Puffer mit bufferIndex bezeichnet. Ist dieser größer oder gleich der Puffergröße, so kann kein neuer Spannungswert gespeichert werden. Anderenfalls kann der zuletzt gemessene Spannungswert lastCaptured an diese Position gespeichert werden. Bedenken Sie, dass im Anschluss daran der bufferIndex inkrementiert werden muss.

Das Beispiel in Listing 1.1 veranschaulicht die Nutzung von Zeigern in diesem Kontext. Weitere Informationen über die Verwendung von Zeigern können dem Kapitel *5.1.7 Zeiger (Pointer)* im *Begleitenden Dokument* entnommen werden.

Neben der Speicherung von Spannungswerten muss die als getStoredVoltage bezeichnete Zugriffsfunktion implementiert werden, die den Zugriff auf die interne Datenbank des ADC-Moduls ermöglicht. Diese Funktion hat die Aufgabe, den an dem übergebenen Index gespeicherten Messwert auszulesen. In dieser Funktion muss zunächst überprüft werden, ob der übergebene Index gültig ist, d.h. ob er unterhalb der aktuellen Position von bufferIndex liegt. Ist dies der Fall, so gibt die Funktion den gewünschten Spannungswert zurück, anderenfalls den Wert 0.

1.2.4 Zusammenfassung

Folgende Übersicht listet einige Funktionen, Typen und Aufgaben auf, die für diesen Versuch relevant sind. Sie kann als Checkliste verwendet werden. Es ist nicht garantiert, dass diese Übersicht vollständig ist.

Aufgaben zur AD/DA-Wandlung

- [ ] Implementierung von `os_input`
- [ ] `void manuell(void)`
  - [ ] Ermöglicht das Ansteuern des R-2R-Netzwerks mit den Schiebereglern
  - [ ] Zeigt den Status der Schieberegler auf den LEDs an
- [ ] `void sar(void)`
  - [ ] Implementierung eines AD/DA-Wandlers nach dem Prinzip eines SAR-Wandlers
  - [ ] Die Wandlung soll erst nach dem Drücken des Buttons starten und zusätzlich auf den LEDs angezeigt werden
- [ ] `void tracking(void)`
  - [ ] Implementierung eines AD/DA-Wandlers nach dem Prinzip eines Tracking-wandlers
  - [ ] Die Wandlung soll erst nach dem Drücken des Buttons starten und zusätzlich auf den LEDs angezeigt werden

Aufgaben zum ATmega644

- [ ] Erweiterung von `os_input` auf vier Taster
- [ ] Programm 1: HelloWorld
  - [ ] Verwendung des LCD-Treibers
  - [ ] Ausgabe des Textes „Hallo Welt!“ auf dem LCD
- [ ] Programm 2: Binäruhr
  - [ ] Implementierung eines Moduls zur Verwaltung der Uhrzeit

- [ ] Implementierung eines Treibers zur Ansteuerung des LED-Bargraphen
- [ ] Anzeige einer Binäruhr auf dem LED-Bargraphen und dem LCD

- [ ] Programm 3: Interner AD-Wandler
  - [ ] Anzeige der gemessenen Spannung auf dem LED-Bargraphen und dem LCD
  - [ ] Implementierung einer Datenbank zur Verwaltung von Messwerten
  - [ ] Umsetzung der Benutzerschnittstelle, die die momentane Messspannung anzeigt sowie das Speichern und Anzeigen von Messdaten ermöglicht

1.3 In Präsenz

Dieser Teil des Versuchs wird in der Präsenzphase durchgeführt. Hier werden die im vorherigen Abschnitt beschriebenen Programme auf der Hardware getestet.

1.3.1 Versuche zur AD/DA-Wandlung

Beide Verfahren zur AD/DA-Wandlung (Tracking und Sukzessive-Approximation) wurden als Hausaufgabe vorbereitet und werden nun auf der Hardware getestet. Dafür benötigen Sie die AD/DA-Platine (Abbildung 1.2). Die Platine ist so konstruiert, dass sie auf das Evaluationsboard aufgesetzt werden kann und somit mit den Ein- und Ausgängen des Mikrocontrollers verbunden ist.

Hinweis

Achten Sie darauf, dass die Anschlüsse der Platine beim Aufsetzen nicht verbogen werden und richtig angeschlossen sind.

! Achtung

Bei aufgesetzter AD/DA-Platine muss die Spannungsversorgung an der Platine und nicht am Evaluationsboard erfolgen (Markierung 3 in Abbildung 1.2).

SA-Wandler

Ändern Sie den Versuchsaufbau nun wie folgt:

! Achtung

1. Begrenzen Sie den Strom des Netzgerätes auf 200 mA (siehe Dokument *Bedienungsanleitungen und Sicherheitshinweise*).
2. Stellen Sie die Spannung des Netzgeräts auf 0 V ein und schalten Sie das Netzgerät wieder aus.
3. Verbinden Sie - vom Ausgang A des Netzgeräts mit Input - (Markierung 5 in Abbildung 1.2) auf der Platine.
4. Verbinden Sie + vom Ausgang A des Netzgeräts mit Input + (Markierung 5 in Abbildung 1.2) auf der Platine. Bei dieser Spannung handelt es sich um die zu messende Spannung (U_{mess}).

Testen Sie den Sukzessive-Approximation-Wandler, den Sie programmiert haben. Überprüfen Sie auf geeignete Art die Funktionsweise.

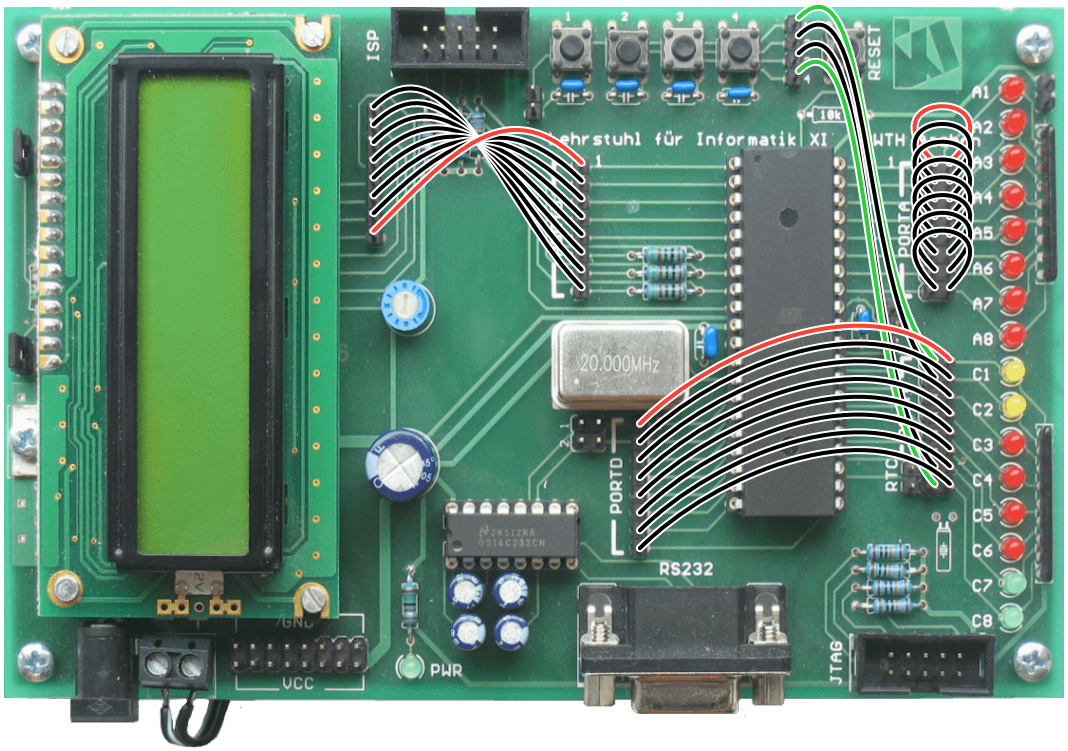


Abbildung 1.12: Verkabelung für die Aufgaben zum ATmega644

Tracking-Wandler

Testen Sie den Tracking-Wandler, den Sie programmiert haben.

1.3.2 Versuche zum ATmega644

Trennen Sie die Spannungsversorgung der AD/DA-Platine sowie des Evaluationsboards. Nehmen Sie die Platine vorsichtig und gleichmäßig vom Evaluationsboard ab. Verkabeln Sie im Anschluss daran das Evaluationsboard entsprechend Pinbelegung 1.2 bzw. Abbildung 1.12.

Programm 1: HelloWorld

Während des Navigierens im Menü muss die korrekte Funktionsweise des Input-Moduls geprüft werden: Das Blättern soll erst beim Loslassen der Taster ausgeführt werden und ein Programm erst dann gestartet werden, wenn Enter gedrückt und losgelassen wurde. Die Ausgabe dieses Programms ist ein gleichmäßiger blinkender Schriftzug „Hallo Welt!“.

Programm 2: Binäruhr

Für dieses Programm müssen die Pins A0 bis A7 mit den nebenliegenden LEDs verbunden werden. Die auf dem Display ausgegebene Uhrzeit muss dem beschriebenen Format entsprechen und mit der binär auf dem LED-Bargraphen ausgegebenen Uhrzeit übereinstimmen.

Programm 3: Interner AD-Wandler

Für den internen AD-Wandler muss die Verbindung des Pins A0 zur nebenliegenden LED entfernt werden. Stattdessen wird Pin A0 mit der Spannungsquelle verbunden. Für diesen Aufbau soll also LED0 nicht verwendet werden.

! Achtung

1. Begrenzen Sie den Strom des Netzgerätes auf 200 mA (siehe Dokument *Bedienungsanleitungen und Sicherheitshinweise*).
2. Stellen Sie die Spannung des Netzgeräts auf 0 V ein und schalten Sie das Netzgerät wieder aus.
3. Verbinden Sie - vom Ausgang A des Netzgeräts mit GND (Stiftleiste neben VCC) auf dem Evaluationsboard.
4. Verbinden Sie + vom Ausgang A des Netzgeräts mit Pin A0 des Evaluationsboards.

! Achtung

Achten Sie darauf, dass die ausgegebene Spannung des Netzgerätes einen Spannungswert von 6 V nicht überschreitet! Zu hohe Spannungen zerstören den Mikrocontroller.

1.3.3 Anwendungsaufgaben

Während des Praktikumstermins werden Aufgaben bereitgestellt, mit denen Sie Ihre im Versuch erlernten Fähigkeiten und Kenntnisse nachweisen. Diese Aufgaben sind eigenständig als Team zu bearbeiten und bis spätestens eine Stunde vor Versuchsende vorzuführen. Die Anwendungsaufgabe für Ihre jeweilige Gruppe finden Sie im Netzlaufwerk im Unterordner **Anwendungsaufgaben**. Wenn Sie die Aufgabe gelöst haben, müssen Sie diese einem Tutor vorzeigen und nach dem Bestehen im Moodle-Lernraum hochladen.

! Achtung

Weiterverbreitung der Aufgaben, sowie das Plagiieren oder Teilen von Lösungen ist **nicht gestattet** und wird als **Täuschungsversuch** gewertet, der zum Ausschluss aus dem Praktikum führt.

1.4 Pinbelegung

Port	Pin	Belegung
B	0	R-2R-Netzwerk (R1)
	1	R-2R-Netzwerk (R2)
	2	R-2R-Netzwerk (R3)
	3	R-2R-Netzwerk (R4)
	4	R-2R-Netzwerk (R5)
	5	R-2R-Netzwerk (R6)
	6	R-2R-Netzwerk (R7)
	7	R-2R-Netzwerk (R8)
D	0	DIP-Schalter 1
	1	DIP-Schalter 2
	2	DIP-Schalter 3
	3	DIP-Schalter 4
	4	DIP-Schalter 5
	5	DIP-Schalter 6
	6	DIP-Schalter 7
	7	DIP-Schalter 8

Port	Pin	Belegung
A	0	LED 1
	1	LED 2
	2	LED 3
	3	LED 4
	4	LED 5
	5	LED 6
	6	LED 7
	7	LED 8
C	0	COMP_OUT
	1	Button
	2	Reserviert für JTAG
	3	Reserviert für JTAG
	4	Reserviert für JTAG
	5	Reserviert für JTAG
	6	LED zur freien Verfügung
	7	LED zur freien Verfügung

Pinbelegung 1.1: Für Versuch 1 – AD/DA-Wandlung

Port	Pin	Belegung
B	0	LCD Pin 0 (D4)
	1	LCD Pin 1 (D5)
	2	LCD Pin 2 (D6)
	3	LCD Pin 3 (D7)
	4	LCD Pin 4 (RS)
	5	LCD Pin 5 (EN)
	6	LCD Pin 6 (RW)
	7	-
D	0	LED 9
	1	LED 10
	2	LED 11
	3	LED 12
	4	LED 13
	5	LED 14
	6	LED 15
	7	LED 16

Port	Pin	Belegung
A	0	LED 1 / Messspannung (ADC)
	1	LED 2
	2	LED 3
	3	LED 4
	4	LED 5
	5	LED 6
	6	LED 7
	7	LED 8
C	0	Button 1: Enter
	1	Button 2: Up
	2	Reserviert für JTAG
	3	Reserviert für JTAG
	4	Reserviert für JTAG
	5	Reserviert für JTAG
	6	Button 3: Down
	7	Button 4: Escape

Pinbelegung 1.2: Für Versuch 1 – ATmega644